

**CLAID: A UNIFIED SOCIAL NETWORK ANALYSIS
FRAMEWORK FOR COMMUNITY-LEVEL
ANOMALY AND INFLUENCER DETECTION
A PROJECT REPORT**

Submitted by

RAMYA S

RUPESH A

AKILAN K

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING



ANNA UNIVERSITY: CHENNAI 600 025

MAY 2024

DEPARTMENT OF COMPUTER TECHNOLOGY

ANNA UNIVERSITY, MIT CAMPUS

CHENNAI – 600044

BONAFIDE CERTIFICATE

Certified that this project report “**CLAID: A Unified Social Network Analysis Framework for Community-Level Anomaly and Influencer Detection**” is the work of **Ramya S (2021503545), Rupesh A (2021503549), Akilan K (2021503719)** in the Creative and Innovative Project Laboratory subject code CS6611 during the period January to May 2024.

SIGNATURE

Dr. P. Pabitha

SUPERVISOR

Associate Professor

Department of Computer Technology

Anna University, MIT Campus

Chrompet – 600044.

SIGNATURE

Dr. K. Kottilingam

COURSE IN-CHARGE

Assistant Professor

Department of Computer Technology

Anna University, MIT Campus

Chrompet – 600044.

ABSTRACT

Social networks, intricate webs of connections among individuals, demand efficient identification of communities for diverse applications. These communities, comprised of individuals with shared connections, form the backbone of network structures. However, not all participants within social networks are genuine, necessitating robust anomaly detection to pinpoint suspicious nodes or activities, such as fake accounts or misinformation spreaders. Additionally, within each community, certain individuals wield more influence than others, shaping the group's dynamics.

Despite the importance of these tasks, prevailing deep learning-based methods for anomaly detection in social networks often fall short. These methods, commonly reliant on graph autoencoders, struggle to fully capitalize on the wealth of network data, resulting in suboptimal performance. Moreover, scalability remains a pervasive issue, particularly concerning large-scale networks. Existing approaches often grapple with high computational demands and hindering their efficiency.

CLAID is an innovative framework that integrates community detection, anomaly detection, and influential user identification, streamlining the analysis of social networks. By combining these tasks into a single system, CLAID enhances

efficiency and effectiveness. It uses advanced algorithms to identify groups of connected individuals, detect suspicious activities like fake accounts or misinformation spreaders, and pinpoint influential users within these groups. With CLAUD, researchers and analysts can gain a comprehensive understanding of social networks, paving the way for more informed decisions and interventions.

ACKNOWLEDGEMENT

We take this humble opportunity to thank Dean, MIT Campus, Anna University, **Dr. Ravichandran Kandaswamy** and **Dr. Jayashree P**, Professor & Head, Department of Computer Technology, MIT Campus, Anna University for providing all the lab facilities in pursuit of this project.

Undertaking this project has helped us learn a lot, and we would like to express our sincere gratitude towards our supervisor **Dr. P Pabitha**, Assistant Professor, Department of Computer Technology, MIT, Anna University, whose kind guidance, and proper directions helped shape this project perfectly. The timely feedback from the supervisor was very instrumental in successful completion of the project work carried out. Constant encouragement to implement the project to the fullest of our potential led us through, during the difficult phase of the project, and assisted in the successful completion of the proposed project work.

We acknowledge and thank the efforts and feedback of the panel members **Dr. S Chitra**, Associate Professor, Department of Computer Technology, MIT, Anna University, **Dr. V P Jayachitra**, Assistant Professor, Department of Computer Technology, MIT, Anna University, and **Dr. K Kottilingam**, Assistant Professor, Department of Computer Technology, MIT, Anna University, in reviewing our work, providing constant valuable comments and encouraging us to view the different aspects of the project in successful implementation of the project.

We thank all the teaching and non-teaching members of the Department of Computer Technology, MIT, Anna University, seniors, juniors, and friends whose appreciable help, either directly or indirectly helped in the smooth completion

of the Creative and Innovative Project in a limited time frame.

Ramya S [2021503545]

Rupesh A [2021503549]

Akilan K [2021503719]

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF FIGURES	xi
1.	INTROUDUCTION	1
	1.1 OVERVIEW	1
	1.2 SOCIAL NETWORK	1
	1.3 COMMUNITY DETECTION IN SOCIAL NETWORK	2
	1.4 ANOMALY DETECTION IN SOCIAL NETWORK	3
	1.5 INFLUENCIAL USER DETECTION IN SOCIAL NETWORK	4
	1.6 OBJECTIVES	4
	1.7 SUMMARY	5
2.	LITERATURE SURVEY	6
	2.1 OVERVIEW	6
	2.2 COMMUNITY DETECTION	6
	2.3 ANOMALY DETECTION	8
	2.4 INFLUENTIAL USER DETECTION	9
	2.5 SUMMARY	11

3.	PROPOSED WORK	13
	3.1 OVERVIEW	13
	3.2 PROPOSED WORK	13
	3.3 MODULES	15
	3.3.1 Community Detection Using Louvain Modularity	16
	3.3.2 Anomaly Detection Using Isolation Forest	17
	3.3.3 Influencer Users' Detection Using Betweenness Centrality	20
	3.4 SUMMARY	21
4.	IMPLEMENTATION	22
	4.1 OVERVIEW	22
	4.2 HARDWARE AND SOFTWARE REQUIREMENTS	22
	4.2.1 Computer	22
	4.2.2 Python	23
	4.3 DATASET DESCRIPTION	23
	4.4 TOOLS REQUIRED	24
	4.4.1 Igraph	24
	4.4.2 Networkx	25
	4.4.3 Matplotlib	25
	4.4.4 Community	25
	4.4.5 Sklearn	26

4.4.6 Flask	26
4.4.7 Gradle Tool	27
4.5 COMMUNITY DETECTION USING LOUVAIN MODULARITY	27
4.5.1 Pseudo Code for Community Detection Using Louvain Modularity	29
4.6 ANOMALY DETECTION USING ISOLATION FOREST	30
4.6.1 Pseudo Code for Anomaly Detection Using Isolation Forest	32
4.7 INFLUENCER USERS' DETECTION USING BETWEENNESS CENTRALITY	34
4.7.1 Pseudo Code for Influencer Users' Detection Using Betweenness Centrality	36
4.8 SUMMARY	38
5. RESULTS AND ANALYSIS	39
5.1 OVERVIEW	39
5.2 RESULTS	39
5.2.1 Community Detection	40
5.2.2 Anomaly Detection	40
5.2.3 Influential user detection	41
5.2.4 CLAID framework	42
5.2.5 Evaluation Metrics	43

5.3 SUMMARY	46
6. CONCLUSION AND FUTURE WORKS	47
6.1 CONCLUSION	47
6.2 FUTURE WORKS	48
7. REFERENCES	49

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1	CLAID FRAMEWORK	5
2	BLOCK DIAGRAM OF CLAUD FRAMEWORK	15
3	ARCHITECTURE OF COMMUNITY DETECTION USING LOUVAIN MODULARITY	17
4	ARCHITECTURE OF ANOMALY DETECTION USING ISOLATION FOREST	19
5	ARCHITECTURE OF INFLUENTIAL USER USING BETWEENNESS CENTRALITY	21
6	DATASET	24
7	CODE SNIPPET OF COMMUNITY DETECTION USING LOUVAIN MODULARITY	28
8	CODE SNIPPET OF ANOMALY DETECTION USING ISOLATION FOREST	31
9	CODE SNIPPET OF INFLUENTIAL USER DETECTION USING BETWEENNESS CENTRALITY	35
10	CLUSTERS FORMED USING LOUVAIN MODULARITY	40
11	ANOMALOUS NODES DETECTED USING ISOLATION FOREST	41

12	INFLUENTIAL NODES DETECTED USING BETWEENNESS CENTRALITY	41
13	CLAID FRAMEWORK USING GRADLE TOOL	42
14	COMPARISON OF COMMUNITY DETECTION METHODS	44
15	COMPARISON OF ANOMALY DETECTION METHODS	45
16	COMPARISON OF INFLUENTIAL USER DETECTION METHODS	46

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The introduction sets the stage for understanding social networks as dynamic online communities where people connect globally. It emphasizes the importance of identifying groups, spotting unusual activities, and recognizing influential users within these networks. The project aims to build a framework that combines various techniques to achieve these goals, making social networks safer and more engaging for users.

1.2 SOCIAL NETWORK

Social networks, the digital ecosystems that have become integral parts of our daily lives, serve as dynamic platforms where individuals, interact, and form virtual communities. These platforms have revolutionized communication, enabling people to connect with others globally, regardless of geographical barriers. At their core, social networks foster the exchange of information, the cultivation of relationships, and the creation of communities centered around shared interests, activities, or relationships.

Whether it's reconnecting with old friends, establishing professional connections, or discovering like-minded individuals, social networks offer a plenty of opportunities for engagement and interaction. However, amidst the vast network of connections lies a complex landscape where authenticity and trust can be challenged. Instances of fake profiles, misinformation, and suspicious activities underscore the need for vigilant monitoring and detection mechanisms to safeguard the integrity and functionality of these platforms.

Moreover, within these interconnected communities, a subtle hierarchy of influence emerges, with certain individuals holding greater power and impact than others. Understanding these dynamics and identifying influential users are essential tasks in unravelling the complex fabric of social networks.

1.3 COMMUNITY DETECTION IN SOCIAL NETWORK

Community detection in social networks is the process of identifying groups of individuals or nodes within a network that are more densely connected to each other than to the rest of the network. These groups, often referred to as communities or clusters, typically share common attributes, interests, or relationships. There are various algorithms and techniques used for community detection, each with its strengths and weaknesses. Some methods focus on identifying densely connected subgraphs, while others consider factors such as node attributes, edge weights, or network dynamics.

One popular approach is modularity optimization, which aims to maximize the modularity of a network partition. Modularity measures the strength of division of a network into communities, with higher values indicating a better division. Algorithms like Louvain Modularity and Newman-Girvan method are commonly used for modularity optimization. Another approach is spectral clustering, which leverages the eigenvalues and eigenvectors of the network's adjacency or similarity matrix to partition the network into communities. Spectral clustering is effective for networks with clear community structures but may struggle with large-scale networks due to computational complexity.

Additionally, there are methods based on random walks, label propagation, and hierarchical clustering, each offering unique advantages depending on the characteristics of the network being analysed. Community detection has numerous applications across various domains, including social network analysis,

recommendation systems, and understanding information diffusion. By uncovering the underlying structure of social networks, community detection algorithms enable researchers and analysts to gain insights into group behaviour, identify key influencers, and understand the dynamics of information flow within communities.

1.4 ANOMALY DETECTION IN SOCIAL NETWORK

Anomaly detection in social networks involves identifying unusual or suspicious activities, behaviours, or patterns that deviate from the norm within the network. These anomalies can range from fake accounts and malicious activities to unexpected changes in user behaviour or network dynamics. Detecting anomalies in social networks is crucial for maintaining the security, and functionality of the platform. It helps to identify and mitigate potential threats such as fake profiles that can harm users and undermine trust in the network.

There are various techniques used for anomaly detection in social networks, including statistical methods, machine learning algorithms, and network analysis approaches. Statistical methods involve analysing patterns in user behaviour or network activity and flagging deviations from expected norms. Machine learning algorithms can be trained on labelled data to recognize patterns associated with anomalies, allowing them to automatically detect suspicious activities. Network analysis approaches leverage the structure and dynamics of the social network itself to identify anomalies. This can involve analysing the connectivity patterns, and community structures within the network to detect unusual clusters.

Anomaly detection in social networks is an ongoing challenge due to the constantly evolving nature of online interactions and the sophistication of malicious actors. However, by leveraging a combination of techniques and

continuously monitoring network activity, platforms can effectively detect and mitigate anomalies to ensure a safe and trustworthy user experience.

1.5 INFLUENCIAL USER DETECTION IN SOCIAL NETWORK

In social networks, influencer users hold significant sway over the opinions, behaviours, and interactions of other users within the network. These individuals possess a considerable following and often have a high degree of engagement with their content, making them influential figures within their respective communities.

In the realm of social network analysis, various algorithms have been developed to detect influential users. One prominent approach is the PageRank algorithm, for ranking web pages in search engine results. PageRank evaluates the importance of a node in a network by considering both the number of incoming links and the quality of those links. In the context of social networks, this translates to assessing the influence of a user based on the number and quality of connections they have within the network. Another widely used algorithm is the HITS (Hyperlink-Induced Topic Search) algorithm, in the context of social networks, authority represents users who are considered highly trustworthy or knowledgeable within a specific topic, while hubs are users who connect to many authoritative sources. These algorithms, along with various machine learning and network analysis techniques, play a crucial role in identifying influential users.

1.6 OBJECTIVES

The main objectives of the project are listed below and Figure 1 gives the overview of the framework.

1. To seamlessly integrate node and edge attributes into the community detection process.
2. To detect the communities in the given social network using Louvain Modularity.
3. To incorporate anomaly detection capabilities to identify suspicious communities and to isolate the anomaly node using isolation forest.
4. To identify the influential user within the communities using Betweenness Centrality.

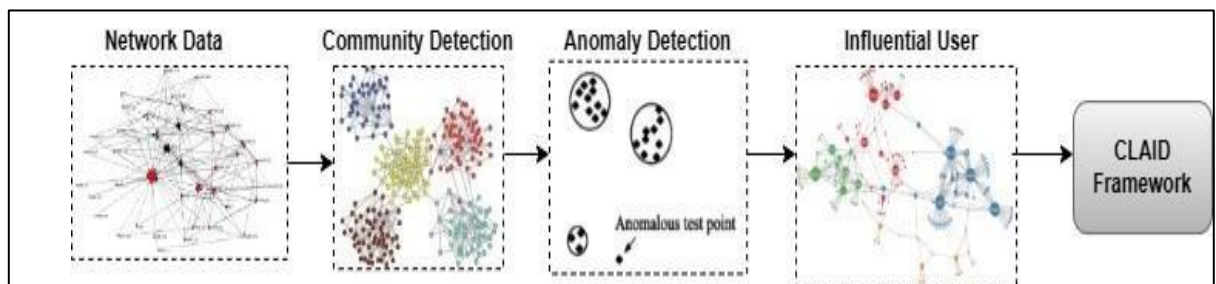


Figure 1 CLAID FRAMEWORK

1.7 SUMMARY

In the subsequent chapters, the project discusses into the intricate details of its objectives, leveraging the foundational understanding established earlier. Each chapter explains a specific aspect crucial to achieving the project's goals, employing a structured approach to address challenges and opportunities. Through comprehensive exploration and analysis, the chapters explain the strategies, methodologies, and potential outcomes, ultimately paving the way for a effective realization of the project's objectives.

CHAPTER 2

LITERATURE SURVEY

2.1 OVERVIEW

The research presented covers diverse methods for understanding social networks. Community detection methods like node2vec and HO-GAT delve into deep learning to uncover closely connected groups of nodes. Anomaly detection techniques like CoLA and label propagation spot unusual behavior by analyzing node relationships. Meanwhile, influential user detection methods like Change Point Detection and NMF-based community detection track changes in community structures and identify influential nodes. These approaches aim to provide deeper insights into network dynamics, enhancing our understanding of social network communities and anomalies.

2.2 COMMUNITY DETECTION

L. Zhang, 2021 [24] uses the node2vec with random walks to traverse the network and capture node relationships. It aims to learn node representations by maximizing the co-occurrence probability of neighboring nodes. This approach allows node2vec to balance exploring a node's immediate neighborhood (like breadth-first search) and venturing further out in the network (like depth-first search), ultimately capturing both local and community-level information within the node's vector representation.

L. Huang et al ,2023 deploys a deep learning model named Hybrid-Order Graph Attention Network (HO-GAT) that captures intricate relationships between nodes and subgraphs within the network [7]. HO-GAT is trained to rebuild the network's structure and data, and anything that it can't rebuild well is flagged as strange.

C. Tu et al, incorporates community information into DeepWalk, improves vertex and community representations by assigning community labels to vertices in walks based on local context and community distribution [20]. This enhancement enhances tasks such as vertex classification and link prediction, particularly for data-rich vertices. By leveraging community structure, the approach offers deeper insights into network dynamics and improves the overall performance of graph-based learning algorithms like DeepWalk.

E. D. Raj, G. Manogaran, 2021 uses Granular Computing-Based Community Detection to build a special network [14] that incorporates both structural connections and node attributes. It calculates Object Community Factor (OCF) to consider how similar a node's attributes are to other nodes in the network, taking into account multiple attributes and their importance. Then, the algorithm sorts nodes by their OCF and assigns them to communities based on a threshold.

S. Souravlas et al, 2023 paper likely employs probabilistic models for community detection in social networks [17]. This approach involves formulating a probabilistic framework where the likelihood of observing the network structure given community assignments is maximized. Methods such as stochastic block models (SBM) or its variants could be utilized to infer the underlying community structure.

C. He, X. Fei, 2022 paper focuses on employing Nonnegative Matrix Factorization (NMF) for community detection in complex networks [6]. NMF decomposes the network adjacency matrix into two matrices representing communities and community memberships, respectively. By enforcing nonnegativity constraints on the matrices, NMF effectively identifies community structures within the network.

W. Luo, 2021 paper likely presents methods for multiscale local community detection in social networks [12]. This approach involves identifying densely connected regions at various scales within the network. Algorithms such as

Louvain method, Infomap, or modularity optimization may be utilized to find communities at different resolutions, allowing for a comprehensive understanding of the network's hierarchical structure.

X. Su et al, 2021 paper provides a comprehensive survey on the integration of deep learning techniques for community detection in social networks [18]. Deep learning methods, such as neural networks, have shown promise in capturing complex patterns in network data and can be adapted to identify community structures more effectively.

2.3 ANOMALY DETECTION

Y. Liu, Z. Li, 2022 proposes CoLA which addresses the challenge of identifying anomalies by looking at the relationships between nodes [11]. It achieves this by introducing a contrastive learning approach that analyzes pairs of a target node and its local subgraph. By comparing these pairs, CoLA can distinguish between normal nodes and anomalies, where anomalies have a weaker connection to their surrounding nodes.

X. Chen, J. Song, 2023 uses Label propagation method [10] that detects anomalous behavior in networks. It adapts by dynamically updating anomaly labels based on network structure. By propagating labels through the network, it identifies groups exhibiting unusual activity, making it effective for detecting anomalies in large-scale networks.

S. Zavrak, 2020 uses Autoencoders and Variational Autoencoders (AE/VAE) neural network techniques to reconstruct data [24]. They assign higher anomaly scores to data that the model has difficulty reconstructing. One-Class Support Vector Machines (OCSVM) learn a boundary around normal data points, flagging anything outside the boundary as anomalous.

C.I. Chang, 2022 uses eigenvector admixture based Sparse component

Separation (EAS) approach [2] with Independent Component Analysis (ICA) to extract anomalies while using sparsity constraints to eliminate noise and focus on the most significant anomalies. Though, these methods excels in detecting the anomalies it struggles to isolate those anomalies from the non-anomalous nodes which hinder there accuracy and performance impromacy.

R. H. Elghanuni, 2019 paper likely discusses anomaly detection techniques in online social networks [3]. Anomaly detection is crucial for identifying unusual behaviors or events in social networks that deviate from normal patterns. The paper may explore various methods, such as statistical approaches or machine learning algorithms, to detect anomalies and mitigate potential risks.

C. Wu, L. Zhang, 2023 likely propose temporal intervention strategies for controlling information cascades in social networks [23]. The methodology may involve developing mathematical models to simulate the spread of information and evaluate the effectiveness of different intervention strategies. Optimization algorithms may be employed to identify optimal timing and target influential nodes for intervention, aiming to minimize the spread of misinformation or maximize the diffusion of desired information.

2.4 INFLUENTIAL USER DETECTION

Zhu, P. Li, 2020 uses Change Point Detection by first capturing the structure of the network at different points in time (snapshots) and assigning a score to each snapshot that reflects both its local and global properties [29]. By analyzing the communities within this network and how they change over time, the method can pinpoint moments where the structure of the original network undergoes a significant shift.

K. Berahmand, 2023 employes NMF-based community detection for attributed networks, combining structural connections and node attributes [1]. This method

constructs a specialized network to integrate both structural and attribute information. Through iterative refinement, it updates a similarity matrix and assigns community memberships to each node, facilitating comprehensive network analysis. By leveraging both structural and attribute data, NMF-based community detection offers a holistic approach to understanding network dynamics and community structures.

N. Hafiene, 2020 uses the KroMFac framework [25] for detecting overlapping communities in incomplete networks by characterizing influential nodes and using regularized nonnegative matrix factorization (NMF). It validates the approach using normalized mutual information (NMI) to demonstrate its effectiveness over baseline methods. But it lacks accuracy when influential nodes are not well-defined or when the network data is significantly incomplete.

X. Luo, 2020 proposes Community Detection in Partially Observable Social Networks [13] which integrates group homophily and individual personality of topics to model network communities. It aims to explain the generation mechanisms of intracommunity and intercommunity links by analyzing real networks and identifying nodes with distinctive personality. However, it may not capture all the community interactions and link generation, as it is based on a set of assumptions and mathematical models. Figure 2.4 summarizes the overall literature survey.

X. Huang et al, 2021 survey likely provides an overview of community detection methods specifically designed for multilayer networks [8]. These methods consider the interconnection between different layers of a network and aim to identify communities that span multiple layers, offering insights into the structural and functional relationships across different network domains.

N. Hafiene, 2020 survey likely focuses on the detection of influential nodes in dynamic social networks [5]. Influential nodes play a significant role in information diffusion, opinion spreading, and network dynamics. The paper may

discuss different metrics and algorithms used to identify influential nodes, considering the evolving nature of social networks over time.

G. Zhang, 2018 paper comprehensively reviews methodologies for detecting information cascades in social networks [26]. It likely analyzes various approaches, including network structure analysis, temporal dynamics examination, and content propagation pattern identification. The review may discuss the strengths and limitations of different detection methods and provide insights into their applicability in different contexts.

M. Wang, 2020 likely employ recurrent neural networks (RNNs) to model the temporal dynamics of information cascades in social networks [21]. The methodology may involve training RNNs on historical data of information diffusion to capture the sequential dependencies and predict future cascade behaviors. Various architectures and training strategies may be explored to optimize the performance of the models in capturing the complex dynamics of information spreading.

A. Smith, 2022 study investigates the temporal dynamics of user influence within information cascades in social networks [16]. The methodology likely involves analyzing user engagement patterns, activity levels, and network centrality measures over time to understand how user influence evolves during the spread of information. Time-series analysis techniques may be employed to uncover trends and patterns in user behavior and influence dynamics.

2.5 SUMMARY

The studies discussed in the research provide a comprehensive view of how different methodologies can be employed to analyze social networks effectively. By utilizing techniques like deep learning, granular computing, and nonnegative matrix factorization, researchers have developed sophisticated tools for community detection, anomaly detection, and influential user identification.

These methods enable a deeper understanding of network structures, dynamics, and behaviors, ultimately contributing to improved network management and decision-making processes. The limitations were identified to propose an organized work for our research, as described in the following elaborative headings.

CHAPTER 3

PROPOSED WORK

3.1 OVERVIEW

The CLAID framework presents a novel solution to the complex challenge of identifying influential users and anomalies within social networks. By enhancing the power of machine learning and deep learning techniques, CLAID offers a comprehensive approach to understanding and analyzing social network dynamics. This framework operates through a structured process: starting with data collection, preprocessing, and division into training and testing sets. Subsequently, the model is trained to recognize patterns of community formation, influential users, and anomalous behaviors. Through a network analysis component, communities within the network are identified and optimized for quality.

3.2 PROPOSED WORK

Our proposed framework CLAID addresses the challenge of spotting influential users and anomalies within social networks. The proposed architecture employs a machine learning with deep learning approach for this purpose.

In the initial phase of the process, data is gathered from the social network, forming the foundation for subsequent analysis. This data is then partitioned into distinct training and testing sets, a critical step in ensuring the reliability and generalizability of the model. Subsequently, the training data undergoes preprocessing, a stage aimed at refining and organizing the information for optimal model performance. Once adequately preprocessed, this data is used into

the model, which undergoes intensive training to discern complex patterns within the social network. These patterns encompass various aspects, including the formation of communities, identification of influential users, and detection of anomalous behaviors.

Following the training process, the model is put to the test using the reserved testing data, evaluating its efficiency in identifying influential users and anomalies within the social network. Additionally, the proposed architecture includes a crucial network analysis component, tasked with identifying communities embedded within the intricate network structure. To enhance the quality of these identified communities, modularity optimization techniques are employed, refining their composition and coherence.

Finally, the model's performance undergoes continuous assessment using the evaluation metrics to measure its proficiency in identifying communities, influential users, and anomalies. This comprehensive evaluation framework enables CLAID not only to detect influential users and anomalies but also to unravel the underlying community structure of the social network. By providing invaluable insights into user behavior and content dynamics, CLAID emerges as a powerful tool for the complex landscape of social interactions.

Figure 2 represents the architecture diagram of the proposed system. During the training phase, the communities are formed and the anomalous nodes and influential nodes are detected. Then, the model is trained and CLAID framework is created.

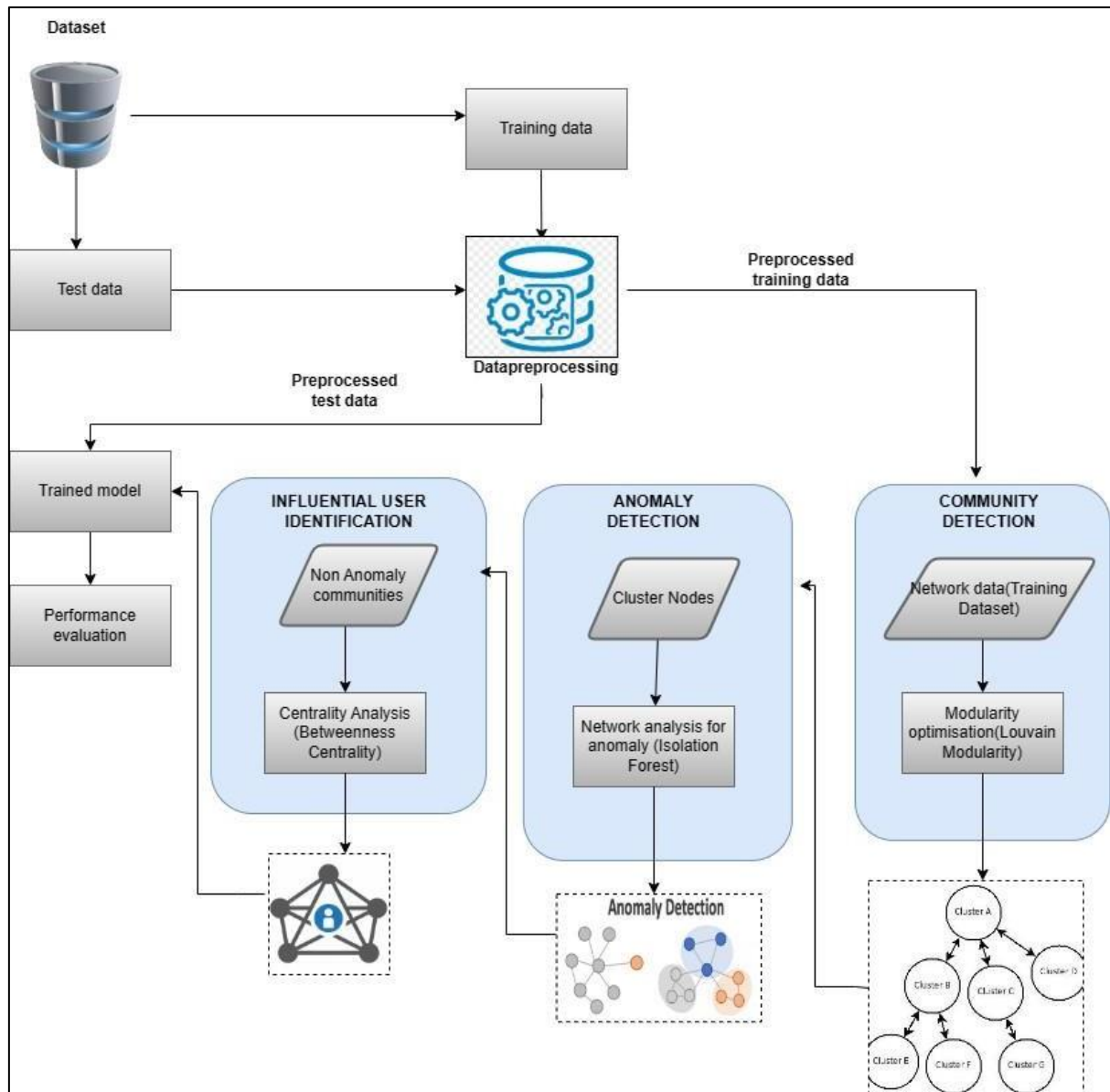


Figure 2 BLOCK DIAGRAM OF CLAID FRAMEWORK

3.3 MODULES

From figure 3.1 it is evident that the proposed system is divided into 3 main modules which are:

- I. Community Detection using Louvain Modularity.
- II. Anomaly Detection using Isolation Forest.
- III. Influential User Detection using Betweenness Centrality.

3.3.1 Community Detection Using Louvain Modularity

Community detection in social networks is a fundamental task of identifying the underlying structure of interconnected nodes. It involves identifying clusters of nodes that exhibit strong internal connections while having fewer connections with nodes outside the cluster. It represents subsets of nodes within the network that share common characteristics or interests.

The process of detecting communities based on Louvain modularity algorithm involves several steps:

1. **Data Preprocessing:** Begin with a network represented as a graph, with nodes representing entities and edges representing relationships. Ensure the data is formatted appropriately for analysis.
2. **Initialization:** Assign each node to its own community, creating an initial partition of the network.
3. **Modularity Optimization:** Iterate through the following steps until no further improvement in modularity is achieved:
4. **Neighbourhood Search:** For each node, evaluate the modularity gain by moving it to a neighbouring community.
5. **Move Node:** Move the node to the community that maximizes the modularity gain, if positive.
6. **Merge Communities:** Merge communities with no loss of modularity, creating a new partition.
7. **Training:** The Louvain Modularity algorithm can be fine-tuned or adjusted based on specific network characteristics or user-defined parameters. Adjustments can be made to enhance performance or adapt to different types of networks.

Figure 3 represents the architecture diagram of community detection using Louvain modularity. It represents the flow of process to form the clusters from the network data.

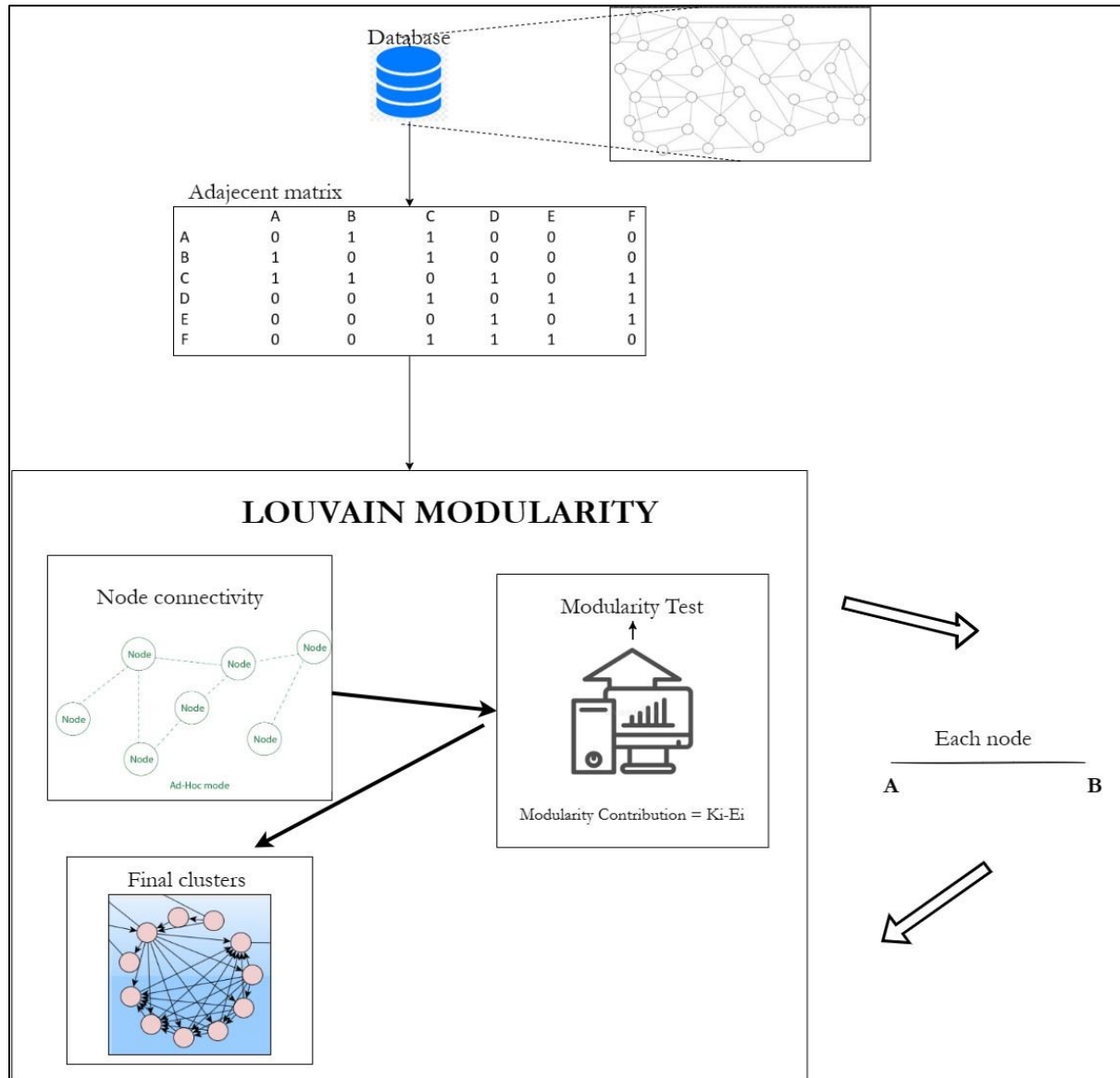


Figure 3 Architecture of Community Detection Using Louvain Modularity

3.3.2 Anomaly Detection Using Isolation Forest

Anomaly detection plays a critical role in identifying suspicious behaviour within social networks, helping to maintain their integrity and functionality. CLAID employs an efficient isolation forest algorithm for anomaly detection, which is

capable of identifying anomalous communities based on their deviation from normal behaviour.

The process of detecting anomalies based on isolation forest algorithm involves several steps:

1. **Data Preparation:** Prepare the network data, ensuring it's structured appropriately for analysis and includes attributes relevant to detecting anomalies.
2. **Community Detection:** Utilize the Louvain Modularity algorithm to partition the network into communities based on connectivity patterns, identifying groups of nodes that are densely interconnected.
3. **Isolation Forest:** Isolation Forest algorithm identifies anomalies by isolating them in the feature space, leveraging the fact that anomalies are typically few in number and have attributes that distinguish them from normal instances.
4. **Anomaly Detection:** Apply the isolation forest algorithm to each node within the detected communities. Evaluate the anomaly score of each node based on its features and position within the feature space.
5. **Anomaly Labelling:** Specify nodes with anomaly scores surpassing a predefined threshold as anomaly nodes. These nodes could represent fake profiles, misinformation spreaders, or other abnormal entities within the network.
6. **Integration with Community Structure:** Associate anomaly labels with the respective communities to provide contextual information about where anomalies occur within the network.
7. **Analysis and Action:** Analyse the isolated anomaly nodes within their community context to understand their impact and devise appropriate actions, such as removal, investigation, or mitigation strategies.

Figure 4 represents the architecture diagram of anomaly detection using isolation forest algorithm. It represents the flow of process to detect the anomaly within each cluster.

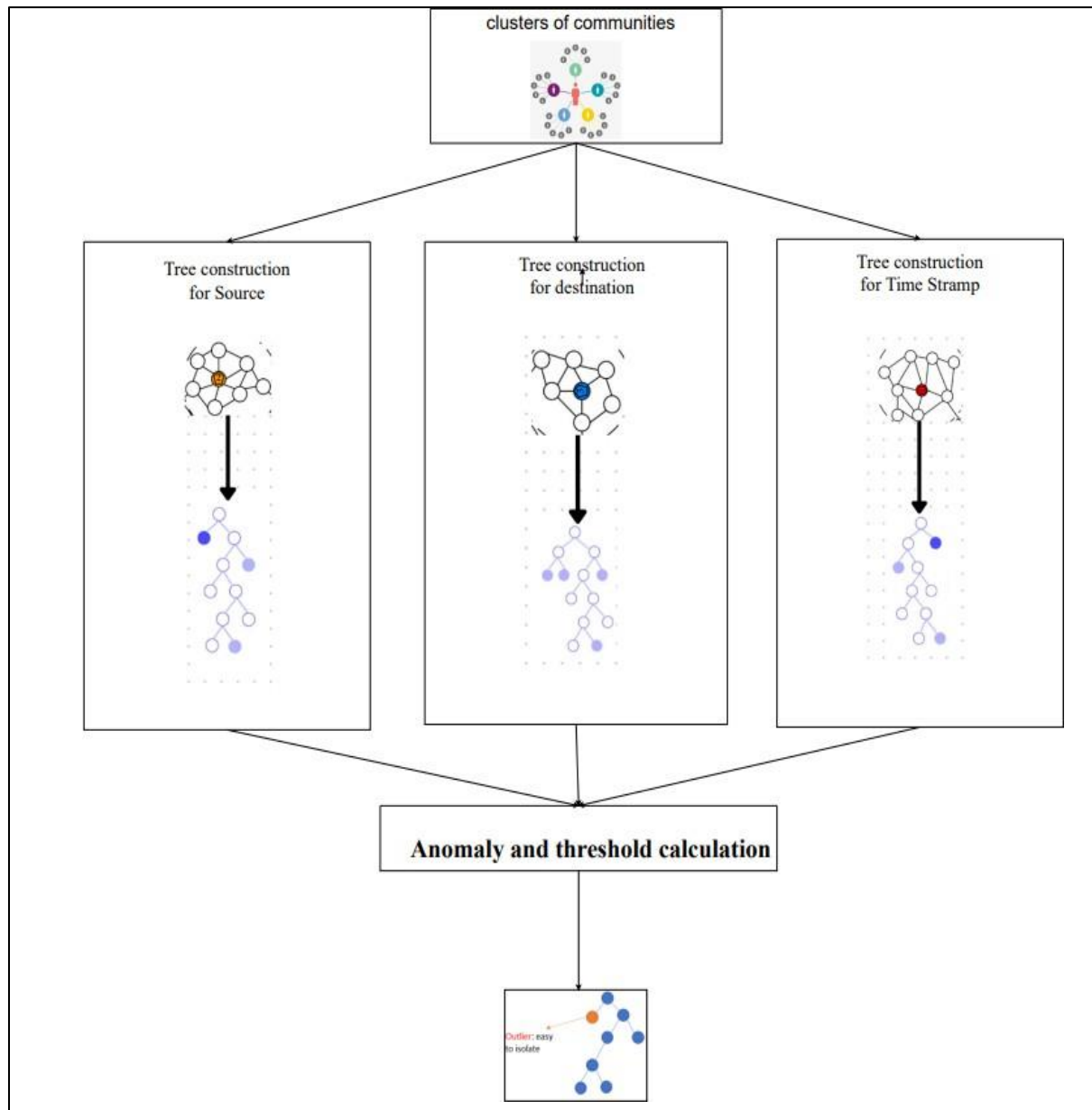


Figure 4 ARCHITECTURE OF ANOMALY DETECTION USING ISOLATION FOREST

3.3.3 Influencer Users' Detection Using Betweenness Centrality

In social networks, identifying influential users within communities is crucial for understanding the dynamics and structure of the network. CLAUD employs a robust algorithm for influential user detection, which aims to pinpoint individuals who wield significant influence within their respective communities.

The process of detecting influential user based on betweenness centrality algorithm involves several steps:

1. **Data Preparation:** Format the non-anomalous community data that is the outcome of the anomaly detection module.
2. **Network Construction:** Create a network graph based on the data, with nodes representing users and edges representing interactions between them.
3. **Betweenness Centrality Calculation:** Compute the betweenness centrality for each user in the network. Betweenness centrality measures the extent to which a node lies on the shortest paths between other nodes.
4. **Identification of Influential Users:** Identify users with high betweenness centrality values, indicating their importance in controlling the flow of information or interactions within the network.
5. **Thresholding:** Apply a threshold to the betweenness centrality values to select only the most influential users or those above a certain percentile. This helps focus on the most significant influencers in the network.
6. **Labeling or Ranking:** Label or rank the identified influential users based on their betweenness centrality scores, indicating their level of influence within the network.

Figure 5 represents the architecture diagram of influential user detection using Betweenness centrality algorithm. It represents the flow of process to detect the influential nodes within each cluster.

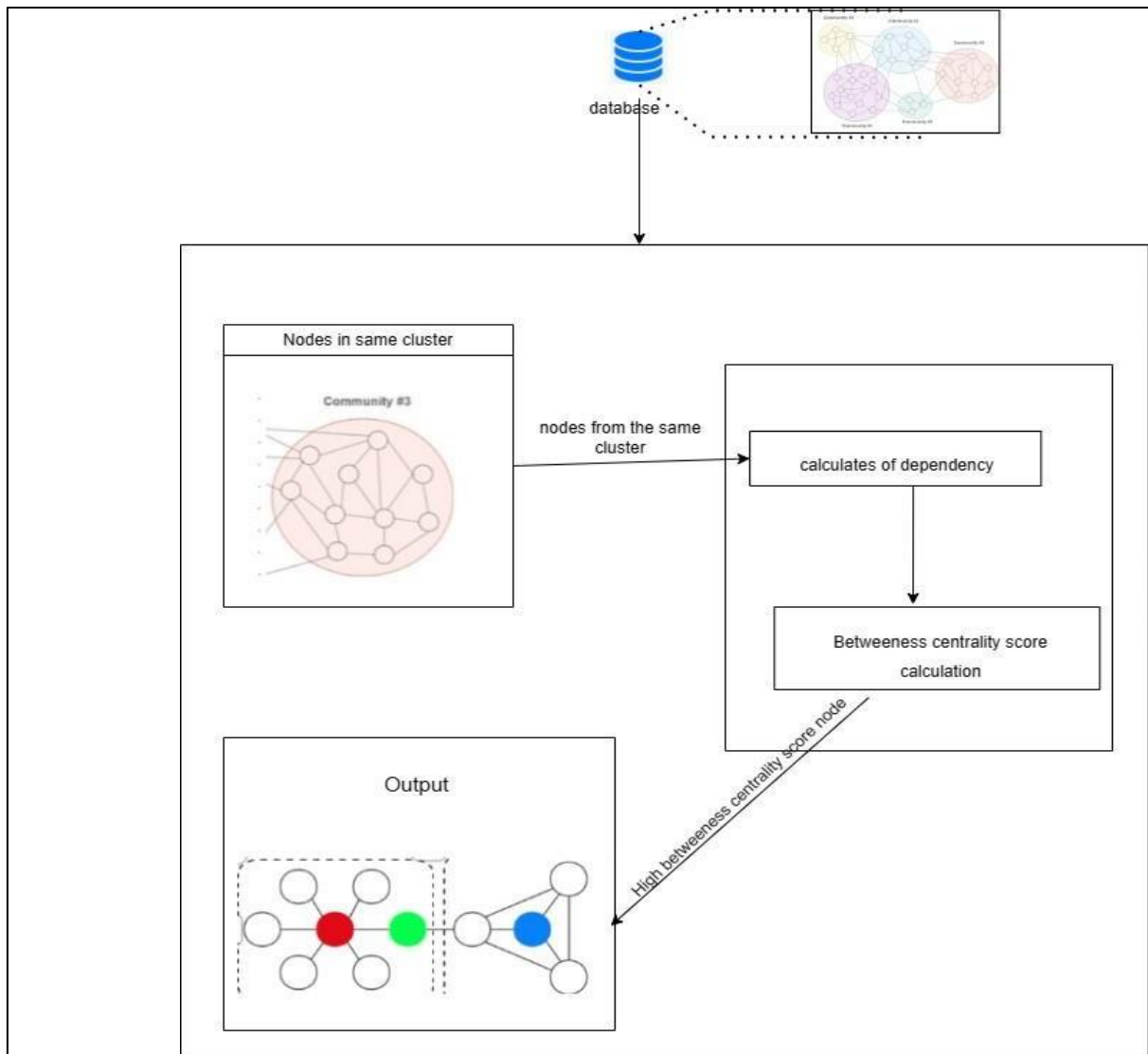


Figure 5 ARCHITECTURE OF INFLUENTIAL USER USING BETWEENNESS CENTRALITY

3.4 SUMMARY

Chapter 3 introduces the proposed work for the CLAID framework, these modules collectively provide comprehensive insights into social network structures, anomalies, and influential figures, facilitating effective network management and decision-making. It highlights the challenges of spotting influential users and anomalies within social networks.

CHAPTER 4

IMPLEMENTATION

4.1 OVERVIEW

The CLAID framework is a comprehensive toolkit designed for analyzing social networks, incorporating a range of models, libraries, and algorithms tailored for this purpose. It utilizes tools like `igraph` and `NetworkX` for machine learning tasks. The framework encompasses three main modules: community detection using Louvain Modularity, anomaly detection using Isolation Forest, and influential user detection using Betweenness Centrality. This chapter gives a deep insight of the implementation of the modules. By integrating these modules, CLAID facilitates the exploration of network structures, identification of anomalies, and detection of influential users, enabling users to gain deeper insights into social network dynamics and make informed decisions regarding network management and optimization.

4.2 HARDWARE AND SOFTWARE REQUIREMENTS

The hardware and software setup required for the CLAID framework are listed for easy understanding and implementation of the modules.

4.2.1 Computer

The hardware requirements for the CLAID framework would typically involve a modern computer with multi-core CPU to expedite the training process, especially considering the computational demands of deep learning algorithms.

A CPU with at least four cores or more would be beneficial. Additionally, having sufficient RAM, typically ranging from 8GB to 16GB or more depending on the size of datasets and model complexity, is essential to accommodate the data and operations efficiently in memory during processing.

4.2.2 Python

The proposed framework, CLAID, utilizes machine learning with deep learning techniques to address the challenge of identifying influential users and anomalies within social networks. For implementation, it requires software tools and programming languages suitable for machine learning and deep learning tasks. Commonly used programming languages for such purposes include Python, due to its extensive libraries like TensorFlow, PyTorch, and scikit-learn which are essential for building and training machine learning models. Additionally, libraries for network analysis such as NetworkX might be used for identifying communities within the network.

4.3 DATASET DESCRIPTION

The dataset used for this project appears to be a collection of posts from various sources (such as subreddits or forums) about different topics. Each entry in the dataset includes information such as the source, target, post ID, timestamp, link sentiment, address, number of followers, phone numbers, number of likes, comments and properties. Totally there are 5000 rows in the dataset. Figure 6 depicts the dataset used for the project.

	A	B	C	D	E	F	G	H	I	J	K	L
1	SOURCE_S	TARGET_S	POST_ID	TIMESTAM	ADDRESS	FOLLOWER	PHONE NC	LIKES	COMMENT	LINK_SENT	PROPERTIES	
2	rddtgamin	rddtrust	1u4pzzs	12/31/201	Chennai	56	9.84E+09	809	167	1	25.0,23.0,0.76,0.0,0.	
3	xboxone	battlefield	1u4tmfs	12/31/201	Madurai	68	8.29E+09	246	205	1	100.0,88.0,0.78,0.0,2	
4	ps4	battlefield	1u4tmos	12/31/201	Coimbatore	64	2.81E+09	175	342	1	100.0,88.0,0.78,0.0,2	
5	fitnesscirc	leangains	1u50xfs	12/31/201	Mumbai	9	5.69E+09	917	218	1	49.0,43.0,0.7755102	
6	fitnesscirc	lifeprotips	1u51nps	12/31/201	Thanjavur	53	7.96E+09	669	471	1	14.0,14.0,0.7857142	
7	cancer	fuckcancer	1u5216s	12/31/201	Madurai	94	2.72E+09	97	425	1	94.0,80.0,0.7765957	
8	jleague	soccer	1u53hzs	12/31/201	Australia	50	6.67E+09	581	99	1	85.0,75.0,0.8,0.0,0.1	
9	bestoftldr	tifu	1u53lxs	#####	Andaman	28	7.34E+09	190	549	1	95.0,78.0,0.7578947	
10	SOURCE_S	TARGET_S	POST_ID	TIMESTAM	punjab	24	4.65E+09	652	520	1	44.0,40.0,0.8409090	
11	rddtgamin	rddtrust	1u4pzzs	12/31/201	Goa	73	1.04E+09	700	514	1	41.0,37.0,0.8048780	
12	xboxone	battlefield	1u4tmfs	12/31/201	Trichy	47	9.77E+09	39	539	1	32.0,27.0,0.75,0.0,0.	
13	ps5	battlefield	1u4tmos	12/31/201	America	99	5.66E+09	358	195	1	69.0,58.0,0.7826086	
14	fitnesscirc	leangains	1u50xfs	12/31/201	Assam	6	6.54E+09	970	231	1	66.0,56.0,0.7878787	
15	fitnesscirc	lifeprotips	1u51nps	12/31/201	Kerala	67	3.93E+09	132	80	1	66.0,59.0,0.6060606	

Figure 6 DATASET

4.4 TOOLS REQUIRED:

The following models, libraries are required for the CLAID framework combining community detection, anomaly detection and influential user detection.

4.4.1 Igraph:

The igraph library is a valuable tool in the realm of network analysis, providing a straightforward interface for working with graph data structures. It's particularly useful when you're dealing with complex networks that represent relationships between various entities. Whether it's social networks, biological networks, or transportation systems, igraph allows you to effortlessly create, manipulate, and analyze these networks. With igraph, you can perform a wide range of tasks, from identifying key nodes in a network to calculating centrality measures that highlight the most influential elements. Its versatility extends to visualization, enabling you to generate clear and informative visual representations of your networks, making it easier to understand their structure and dynamics.

4.4.2 Networkx

NetworkX is another powerful library for working with graphs and networks in Python. It provides a user-friendly interface for creating, manipulating, and analyzing graph structures, making it a valuable tool for various applications in data analysis, social network analysis, and more. With NetworkX, you can effortlessly construct graphs from data, whether it's representing connections between individuals in a social network or relationships between nodes in a biological network. The library offers a wide range of algorithms and functions for tasks such as finding shortest paths, calculating centrality measures, and detecting communities within a network, empowering users to gain valuable insights from their data.

4.4.3 Matplotlib

Matplotlib is a versatile and widely-used Python library for creating static, animated, and interactive visualizations. It offers a comprehensive set of tools for generating plots, charts, and graphs to visualize data in various formats. Whether you're analyzing data, exploring trends, or communicating results, Matplotlib provides the flexibility and customization options needed to create publication-quality visualizations. One of the strengths of Matplotlib is its ability to produce high-quality graphics suitable for both print and digital media. Matplotlib provides the tools necessary to create visually appealing and informative visualizations.

4.4.4 Community

The community library, often used in conjunction with NetworkX, is a Python package specifically designed for detecting communities or clusters within

networks. Communities are groups of nodes within a network that are more densely connected to each other than to nodes outside of the group. These communities can represent meaningful structures within the network, such as social groups in a social network or functional modules in a biological network. The community library provides various algorithms for community detection, including popular methods like the Louvain algorithm and the Girvan-Newman algorithm.

4.4.5 Sklearn

Scikit-learn, often abbreviated as sklearn, is a comprehensive machine learning library for Python that provides efficient tools for data analysis and modeling. It offers a wide range of algorithms and utilities for tasks such as classification, regression, clustering, dimensionality reduction, and more. With scikit-learn, you can easily preprocess and transform your data, train machine learning models, and evaluate their performance using various metrics and techniques. Its user-friendly interface and extensive documentation make it accessible to both beginners and experienced machine learning practitioners, empowering them to build and deploy machine learning solutions for a variety of applications.

4.4.6 Flask

Flask is a lightweight and versatile web framework for Python, designed to make it easy to build web applications and APIs. With Flask, you can quickly create web services, ranging from simple microservices to full-fledged web applications, using Python's simplicity and flexibility. It provides essential features for web development, including routing, request handling, and templating, while allowing developers the freedom to choose their preferred tools and libraries for other aspects of their projects. Flask's minimalist design and

modular architecture make it an excellent choice for developers seeking a straightforward and customizable framework for web development in Python.

4.4.7 Gradle Tool

Gradle is a build automation tool that is popularly used for building, testing, and deploying software projects. It is designed to be flexible, efficient, and highly customizable, making it suitable for a wide range of projects and environments. One of Gradle's key features is its build script DSL (Domain Specific Language), which allows users to define their build logic using a concise and expressive syntax. Gradle uses a concept called "tasks" to organize the build process. Tasks represent individual units of work, such as compiling source code, running tests, or creating a deployable artifact. These tasks can be configured and customized to suit the specific requirements of a project.

4.5 COMMUNITY DETECTION USING LOUVAIN MODULARITY

The proposed Louvain Modularity algorithm addresses the challenge of community detection in social networks, where the network is represented as an adjacency matrix and the goal is to partition the nodes into distinct communities. The algorithm begins by initializing each node as its own community, setting up the groundwork for subsequent analysis. It iteratively evaluates the modularity of potential community structures and reassigns nodes to optimize the overall modularity score. It calculates the degree and expected degree of each node, identifies the best community for each node based on modularity gain, and updates the community assignments accordingly. This iterative process continues until no further improvement in modularity is observed.

Finally, the algorithm returns a list of communities, where each element corresponds to the community ID of the respective node. By iteratively refining the community structure based on modularity optimization, the algorithm offers

a systematic approach to uncovering meaningful community structures within social networks, facilitating deeper understanding in social networks and user interactions.

```
import networkx as nx
import community as community_louvain
import matplotlib.pyplot as plt
import pandas as pd
%matplotlib inline
dataset_path = "/content/dataset100.csv"
df = pd.read_csv(dataset_path)
G = nx.from_pandas_edgelist(df, 'SOURCE_SUBREDDIT', 'TARGET_SUBREDDIT')
plt.figure(figsize=(10, 10))
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_color='skyblue', node_size=1000, edge_color='black', linewidths=1, font_size=15)
plt.title("Network Graph")
plt.show()
partition = community_louvain.best_partition(G)
plt.figure(figsize=(10, 10))
pos = nx.spring_layout(G)
count = len(set(partition.values()))
colors = [plt.cm.tab10(i / count) for i in range(count)]
for i, com in enumerate(set(partition.values())):
    list_nodes = [nodes for nodes in partition.keys()
                  if partition[nodes] == com]
    nx.draw_networkx_nodes(G, pos, list_nodes, node_size=20, node_color=colors[i])
nx.draw_networkx_edges(G, pos, alpha=0.5)
plt.title("Network Graph with Communities (Louvain)")
plt.show()
```

Figure 7 CODE SNIPPET OF COMMUNITY DETECTION USING LOUVAIN MODULARITY

Figure 7 depicts the implementation of Louvain modularity for detecting communities. It iteratively optimizes a metric called modularity, which measures how well nodes are divided into these communities. In each step, nodes are moved between communities to increase modularity. This process continues until a local maximum modularity is reached, revealing the community structure of the network.

4.5.1 Pseudo Code for Community Detection Using Louvain Modularity

- **Input:** A network represented as an adjacency matrix A and the total number of edges m .
- **Output:** A list of communities C , where each element $C[i]$ represents the community ID for node i .

```
1. function louvain(A, m):  
2.   Step 1: Initialize each node to its own community  
3.   C = list(range(len(A)))  
4.   Step 2: Loop until no improvement occurs  
5.   while True:  
6.     improvement = False  
7.     for i in range(len(A)):  
8.       ki = sum(A[i])  
9.       ei = m * sum(A[i]) / sum(sum(A))  
10.      best_community = C[i]  
11.      best_modularity_gain = 0  
12.     for c in neighbors(i):  
13.       updated_ki, updated_ei = update_degree_and_expected_degree(i, c, A)  
14.       Qi = modularity_contribution(i, c, A, updated_ki, updated_ei)  
15.       modularity_contribution = expected_degree - updated_degree  
16.     if Qi > best_modularity_gain:  
17.       best_community = c
```

```

18.    best_modularity_gain = Qi
19.if best_modularity_gain > 0:
20.    C[i] = best_community
21.    update_degree_and_expected_degree(i,    best_community,    A)
        improvement = True
22.if not improvement:
23.    break
24.return C

```

4.6 ANOMALY DETECTION USING ISOLATION FOREST

Anomaly detection plays a critical role in identifying suspicious behavior within social networks, helping to maintain their integrity and functionality. CLAID employs an efficient isolation forest algorithm for anomaly detection, which is capable of identifying anomalous communities based on their deviation from normal behaviour.

The anomaly detection process begins by inputting a set of communities into the isolation forest algorithm. These communities represent interconnected groups of individuals within the social network. The algorithm then proceeds to partition each community into anomalous and non-anomalous subsets, assigning anomaly scores to each community based on its deviation from the norm. The algorithm operates by recursively splitting the data into subsets using randomly chosen features and thresholds. This process continues until each subset contains only a single data point or reaches the specified maximum depth. An average path length is calculated for each data point within the forest of decision trees, representing the number of edges traversed to reach the data point's terminal node.

Finally, the anomaly score for each community is calculated by averaging the path lengths across all decision trees in the forest. This score is normalized by dividing it by the maximum average path length expected for a data point in the dataset. Communities with higher anomaly scores are considered more anomalous, indicating a higher likelihood of containing suspicious activity.

```
import pandas as pd
import networkx as nx
import community as community_louvain
import matplotlib.pyplot as plt
from sklearn.metrics import f1_score
%matplotlib inline
dataset_path = "/content/Dataset.csv"
df = pd.read_csv(dataset_path)
G = nx.from_pandas_edgelist(df, 'SOURCE_SUBREDDIT', 'TARGET_SUBREDDIT')
degree_centrality = nx.degree_centrality(G)
mean_degree_centrality = sum(degree_centrality.values()) / len(degree_centrality)
std_degree_centrality = sum([(x - mean_degree_centrality) ** 2 for x in degree_centrality.values()]) / len(degree_centrality)
threshold = mean_degree_centrality + 2 * std_degree_centrality
anomalies = {node: centrality for node, centrality in degree_centrality.items() if centrality > threshold}
plt.figure(figsize=(12, 12))
pos = nx.spring_layout(G, k=0.15)
nx.draw_networkx_edges(G, pos, edge_color='gray', width=0.5)
nx.draw_networkx_nodes(G, pos, nodelist=[node for node in G.nodes() if node not in anomalies],
                        node_color='lightblue', node_size=100, alpha=0.8)
nx.draw_networkx_nodes(G, pos, nodelist=anomalies.keys(), node_color='red', node_size=300)
nx.draw_networkx_labels(G, pos, labels={}, font_size=12)
plt.title("Network Graph with Anomalies")
plt.show()
anomalies_df = pd.DataFrame.from_dict(anomalies, orient='index', columns=['Degree Centrality'])
predicted_anomalies = {node: 1 if centrality > threshold else 0 for node, centrality in degree_centrality.items()}
y_true = [ground_truth_anomalies.get(node, 0) for node in G.nodes()]
y_pred = [predicted_anomalies.get(node, 0) for node in G.nodes()]
f1score = f1_score(y_true, y_pred)
print("F1 Score:", f1score)
print("Anomalies:")
print(anomalies_df)
```

Figure 8 CODE SNIPPET OF ANOMALY DETECTION USING ISOLATION FOREST

Figure 8 depicts the implementation of isolation forest algorithm to detects the anomalous nodes within the communities. It calculates the degree centrality for each node in the graph and defines a threshold based on the mean and standard deviation of the degree centrality. Nodes with a degree centrality exceeding the threshold are considered anomalies.

4.6.1 Pseudo Code for Anomaly Detection Using Isolation Forest

Input: set of preprocessed datasets.

Output: set of anomalous and non-anomalous communities with anomaly scores.

```
1. def split_node(X_subsample, depth):
2.     if len(X_subsample) == 1:
3.         return Node(depth=depth)
4.     else:
5.         n_samples, n_features = X_subsample.shape
6.         f = np.random.randint(0, n_features)
7.         threshold = np.random.uniform(X_subsample[:, f].min(),
            X_subsample[:, f].max)
8.         left_indices = np.where(X_subsample[:, f] < threshold)[0]
9.         right_indices = np.where(X_subsample[:, f] >= threshold)[0]
10.        left = split_node(X_subsample[left_indices], depth + 1)
11.       right = split_node(X_subsample[right_indices], depth + 1)
12.       return Node(feature_index=f, threshold=threshold, left=left,
            right=right, depth=depth)
13. def average_path_length(x, tree):
14.     node = tree
15.     path_length = 0
16.     while isinstance(node, Node):
```

```

17.     if x[node.feature_index] < node.threshold:
18.         node = node.left
19.     else:
20.         node = node.right
21.     path_length += 1
22. return path_length

24. def isolation_forest(X, n_trees, max_depth):     n_samples = X.shape[0]
25.     anomaly_scores = np.zeros(n_samples)
26.     trees = []
27.     for i in range(n_trees):
28.         indices = np.random.choice(n_samples, n_samples, replace=True)
29.         X_subsample = X[indices]
30.         tree = split_node(X_subsample, 0)
31.         trees.append(tree)
32.         for j in range(n_samples):
33.             anomaly_scores[j] += average_path_length(X[j], tree)
34.     anomaly_scores /= n_trees

35. avg_path_length_max_depth = 2 * (np.log(n_samples - 1) +
    0.5772156649) - (2 * (n_samples - 1) / n_samples)
36.     anomaly_scores /= avg_path_length_max_depth
37. return anomaly_scores

```

4.7 INFLUENCER USERS' DETECTION USING BETWENNESS CENTRALITY

In social networks, identifying influential users within communities is crucial for understanding the dynamics and structure of the network. CLAID employs a robust algorithm for influential user detection, which aims to pinpoint individuals who wield significant influence within their respective communities.

The influential user detection process begins by inputting non-anomalous communities into the algorithm. These communities represent interconnected groups of individuals within the social network. The algorithm then iterates through each community, calculating the betweenness centrality measure for each node to determine its influence within the community.

The algorithm computes the shortest paths between all pairs of nodes within the community graph, utilizing a breadth-first search approach. It then calculates various metrics, including dependency and sigma, to quantify the influence of each node. The betweenness centrality of each node is subsequently determined based on these metrics, representing its overall influence within the community.

Finally, the algorithm identifies the most influential user within each community based on their betweenness centrality scores and compiles a list of influential users for each community.

```

import networkx as nx
import matplotlib.pyplot as plt
from sklearn.metrics import f1_score
from networkx.algorithms import community
G = nx.Graph()
G.add_edges_from([(1, 2), (2, 3), (3, 4), (4, 1)])
colors = ["#00C98D", "#5030C0", "#50F0F0", 'yellow']
pos = nx.spring_layout(G)
lst_m = community.greedy_modularity_communities(G)
color_map_b = {}
keys = G.nodes()
values = "black"
for i in keys:
    color_map_b[i] = values
ground_truth_labels = {1: 0, 2: 0, 3: 1, 4: 1}
predicted_labels = []
for node in G.nodes():
    for i, community in enumerate(lst_m):
        if node in community:
            predicted_labels.append(i)
            break
f1 = f1_score(list(ground_truth_labels.values()), predicted_labels, average='weighted')
print("F1 Score:", f1)
betweenness_centralities = {}
for i, community in enumerate(lst_m):
    community_subgraph = G.subgraph(community)
    betweenness_centralities[i] = nx.betweenness centrality(community_subgraph)
for i, community in enumerate(lst_m):
    community_subgraph = G.subgraph(community)
    node_colors = [betweenness_centralities[i][node] for node in community_subgraph.nodes()]
    nx.draw_networkx_edges(G, pos)
    nodes = nx.draw_networkx_nodes(G, pos, node_color=node_colors, cmap=plt.cm.Blues)
    nx.draw_networkx_labels(G, pos)
    plt.title(f"Community {i} Betweenness Centrality")
    plt.colorbar(nodes, label='Betweenness Centrality')
    plt.axis("off")
    plt.show()

```

**Figure 9 CODE SNIPPET OF INFLUENTIAL USER DETECTION
USING BETWEENNESS CENTRALITY**

Figure 9 depicts the implementation of Betweenness centrality algorithm to detect influential user within each community. Betweenness centrality scores for each node within the community is calculated and the highest centrality score nodes is detected as influential node.

4.7.1 Pseudo Code for Influencer Users' Detection Using Betweenness Centrality

Input: Non anomalous communities.

Output: List of influential users in each community.

```
1. def influential_users_in_communities(communities):
2.     influential_users = []
3.     for community_graph in communities:
4.         num_shortest_paths = {}
5.         dependency = {}
6.         sigma = {}
7.         betweenness centrality = {}
8.         for node in community_graph.nodes:
9.             num_shortest_paths[node] = 0
10.            dependency[node] = 0
11.            sigma[node] = 0 betweenness centrality[node] = 0
12.        for source in community_graph.nodes:
13.            S = deque()
14.            P = {}
15.            sigma = dict.fromkeys(sigma, 0)
16.            sigma[source] = 1
17.            D = {}
18.            for node in community_graph.nodes:
19.                D[node] = float('inf')
20.            D[source] = 0
```

```

21.     Q = deque([source])
22.     while Q:
23.         v = Q.popleft()
24.     S.append(v)
25.         for w in community_graph.neighbors(v):
26.     if D[w] == float('inf'):
27.         Q.append(w)
28.         D[w] = D[v] + 1
29.     if D[w] == D[v] + 1:
30.         sigma[w] += sigma[v]
31.         if w not in P:
32.             P[w] = [v]
33.         else:
34.             P[w].append(v)
35.     while S:
36.         w = S.pop()
38.         for v in P.get(w, []): dependency[v] += (sigma[v] / sigma[w]) *
(1 + dependency[w])
39.         if w != source:
40.             betweenness centrality[w] += dependency[w]
41. for node in community_graph.nodes:
42.     betweenness centrality[node] /= 2

```

```
43. most_influential_user = max(betweenness centrality,  
    key=betweenness centrality.get)  
  
44.    influential_users.append(most_influential_user)  
  
45.    return influential_users
```

4.8 SUMMARY

Chapter 4 incorporates various models, libraries, and algorithms to facilitate in-depth exploration of network structures and behaviors. From community detection using Louvain Modularity to anomaly detection employing Isolation Forest, and influential user detection through Betweenness Centrality, CLAID covers key aspects of social network analysis. By integrating these modules, CLAID provides users with the tools necessary to uncover hidden patterns, identify anomalies, and understand the influence dynamics within social networks. This holistic approach equips users with valuable insights for effective network management and decision-making.

CHAPTER 5

RESULTS AND ANALYSIS

5.1 OVERVIEW

This chapter offers detailed insights into the implemented modules, highlighting the diverse methodologies employed for social network analysis. Through the integration of these modules, the CLAID framework enables users to delve deep into network structures, detect anomalies, and identify influential entities, thereby enhancing their understanding of social network behaviours and facilitating informed decision-making.

5.2 RESULTS

The implemented modules are tested for various qubits, and the results have been documented.

1. The clusters are formed and the communities are detected for the given dataset using the Louvain modularity algorithm.
2. The anomalous nodes in the formed clusters are analysed and isolated using the isolation forest algorithm.
3. From the non- anomalous communities, the influential users are identified using the betweenness centrality algorithm.
4. The evaluation metrics like precision, recall and f1 score for each algorithm is calculated and compared.
5. The algorithms are combined together to form a CLAID framework.

5.2.1 Community Detection

Figure 10 illustrates clusters derived from the dataset through the Louvain modularity algorithm, indicating community structures within the data. The text labels in the image represent groups of nodes, which are connected by edges. These groups are likely communities identified by the Louvain Modularity algorithm. Analysis of these clusters can unveil patterns and relationships, offering insights into the dataset's underlying structures and organizations.

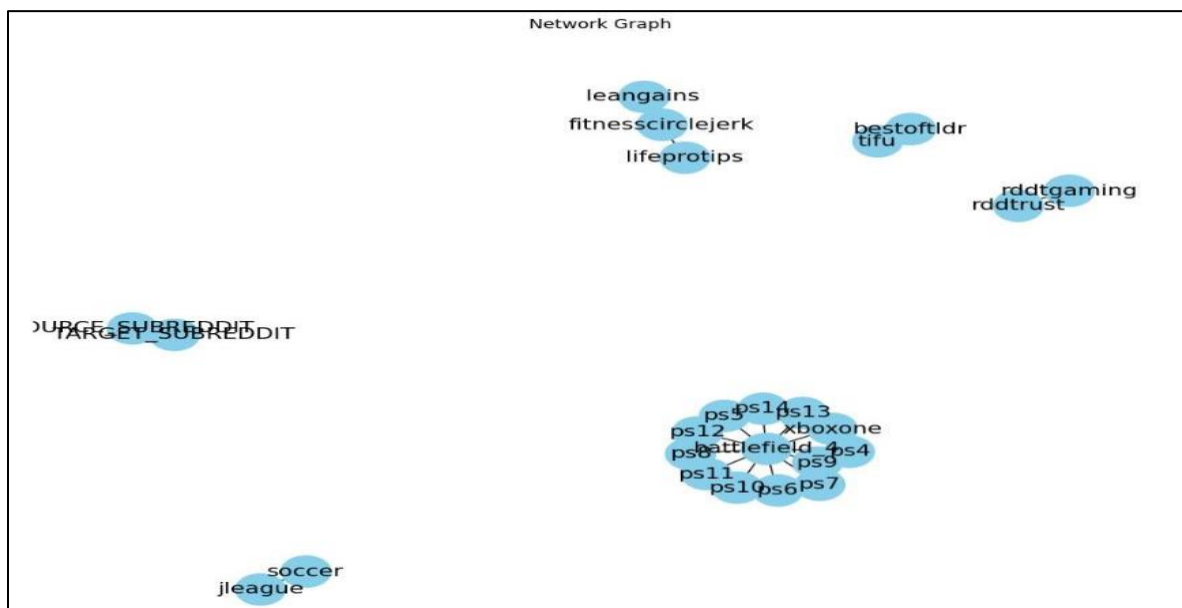
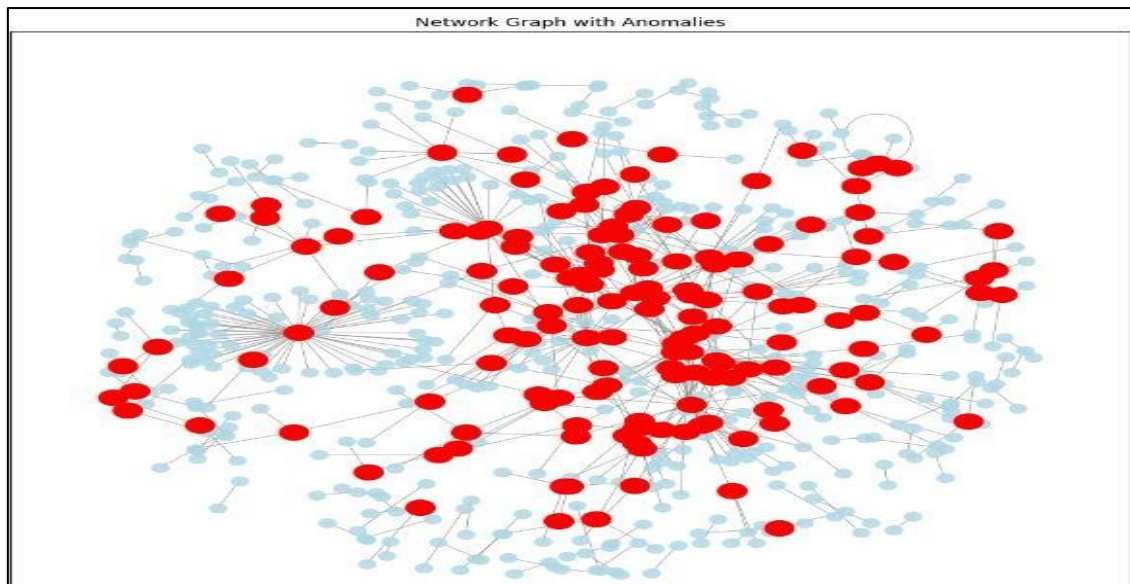


Figure 10 CLUSTERS FORMED USING LOUVAIN MODULARITY

5.2.2 Anomaly Detection

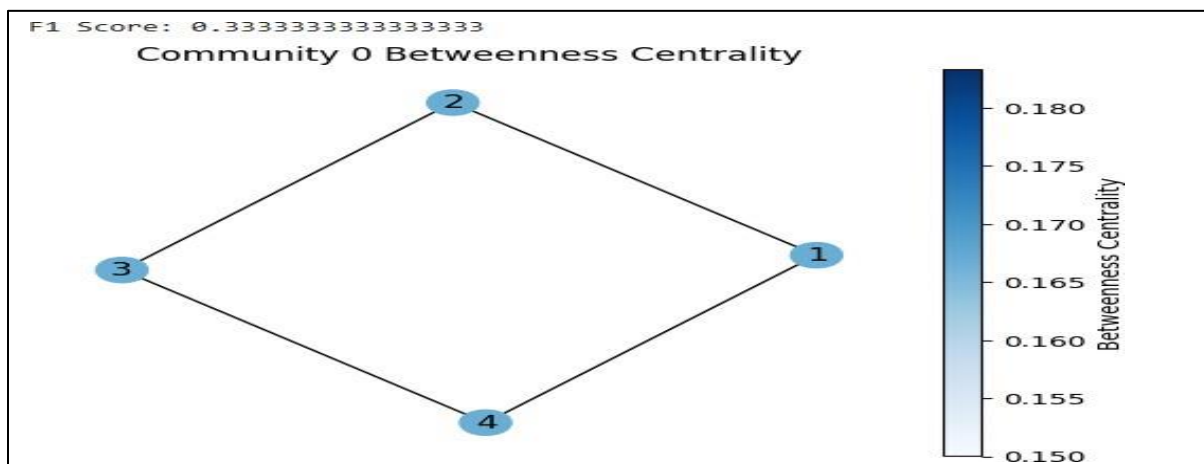
Figure 11 displays the anomalous and non-anomalous nodes that are formed using the isolation forest algorithm. The algorithm verifies the interaction between each node and its edges within the community. The anomalous node patterns are identified and it is verified with the nodes that are newly detected in the community for anomaly.



**Figure 11 ANOMALOUS NODE DETECTED USING
ISOLATION FOREST**

5.2.3 Influential User's Detection

Figure 12 represents the influential nodes that are detected using the betweenness centrality algorithm. The algorithm calculates the betweenness centrality scores for each node in the communities. The highest betweenness centrality scores calculated is detected as the influential user of that community.



**Figure 12 INFLUENTIAL NODES DETECTED USING BETWEENNESS
CENTRALITY**

5.2.4 CLAID Framework

Figure 13 represents the CLAID framework that is created using the gradle tool. The framework lists the communities that are in interaction with the source node and the detected anomaly and influential nodes are displayed in the graph as communities.

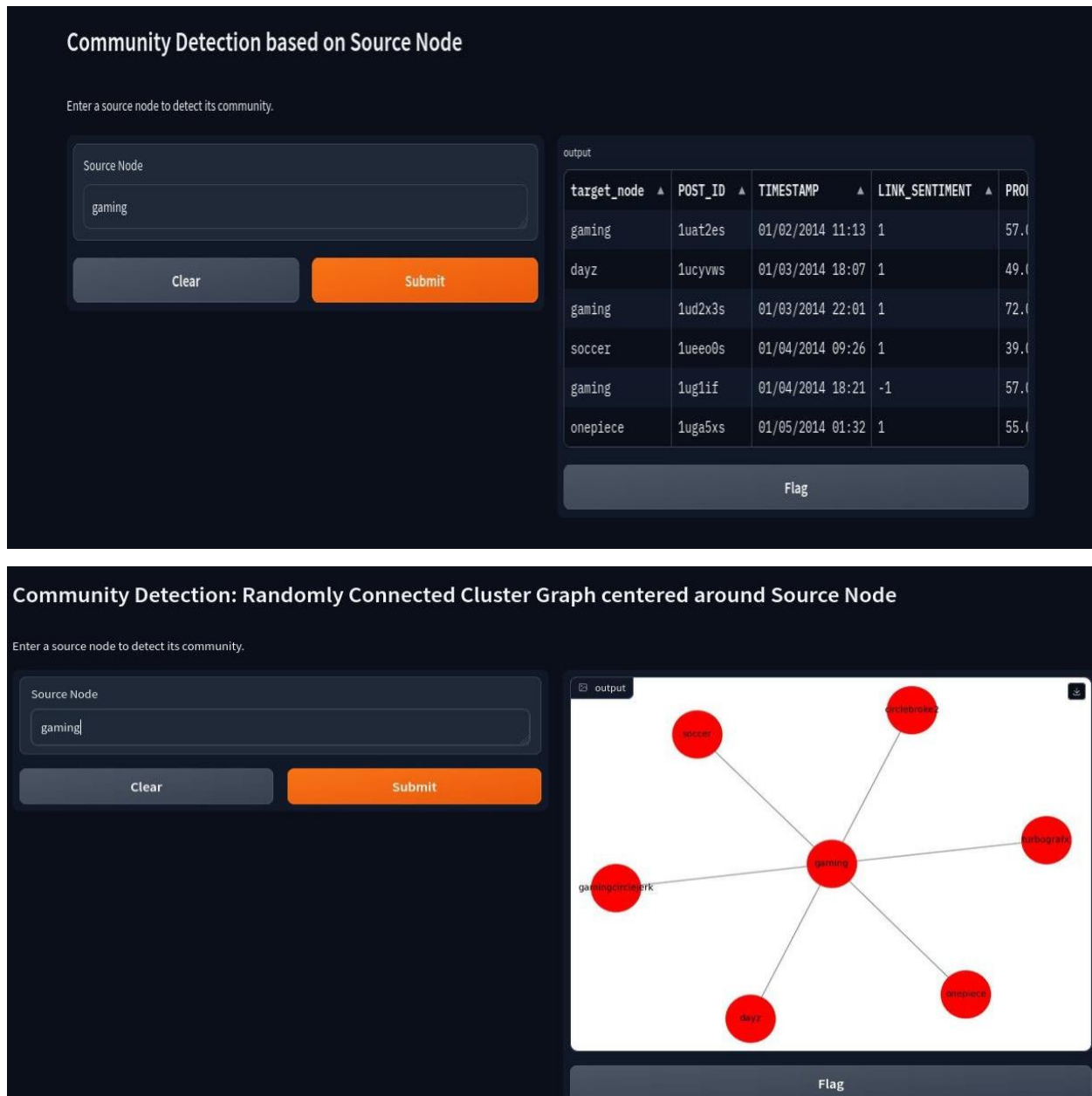


Figure 13 CLAID FRAMEWORK USING GRADLE TOOL

5.2.5 Evaluation Metrics

Precision, recall, and the F1 score are fundamental evaluation metrics used to assess the performance of models, particularly in tasks involving classification, such as identifying influential users and anomalies within social networks. Precision measures the accuracy of positive predictions made by the model, indicating the proportion of correctly identified instances among all instances classified as positive. A high precision signifies that the model's positive predictions are reliable, with a low false positive rate, thus minimizing the chances of incorrectly labeling non-influential users or normal behaviors as influential or anomalous.

Recall assesses the model's ability to detect all relevant instances, evaluating how many actual influential users and anomalies were correctly identified among all existing instances in the social network. A high recall indicates comprehensive coverage of true positives, minimizing false negatives.

The F1 score, a harmonic mean of precision and recall, balances both metrics and provides a single value to gauge model performance, especially useful in imbalanced class distributions like identifying influential users and anomalies in social networks.

Figure 14 shows the visualization and tabulation of evaluation metrics for various community detection methods. The performance metrics of Louvain modularity algorithm is better than other methods infer that the community are formed more accurately using the Louvain modularity algorithm.

Method	Precision	Recall	F1 Score
Nonnegative matrix factorization(NMF)	0.6	0.6	0.5733
DeepWalk	0.5667	0.6	0.5533
Granular Computing	0.4	0.5	0.44
Louvain Modularity	0.7	0.8	0.7333

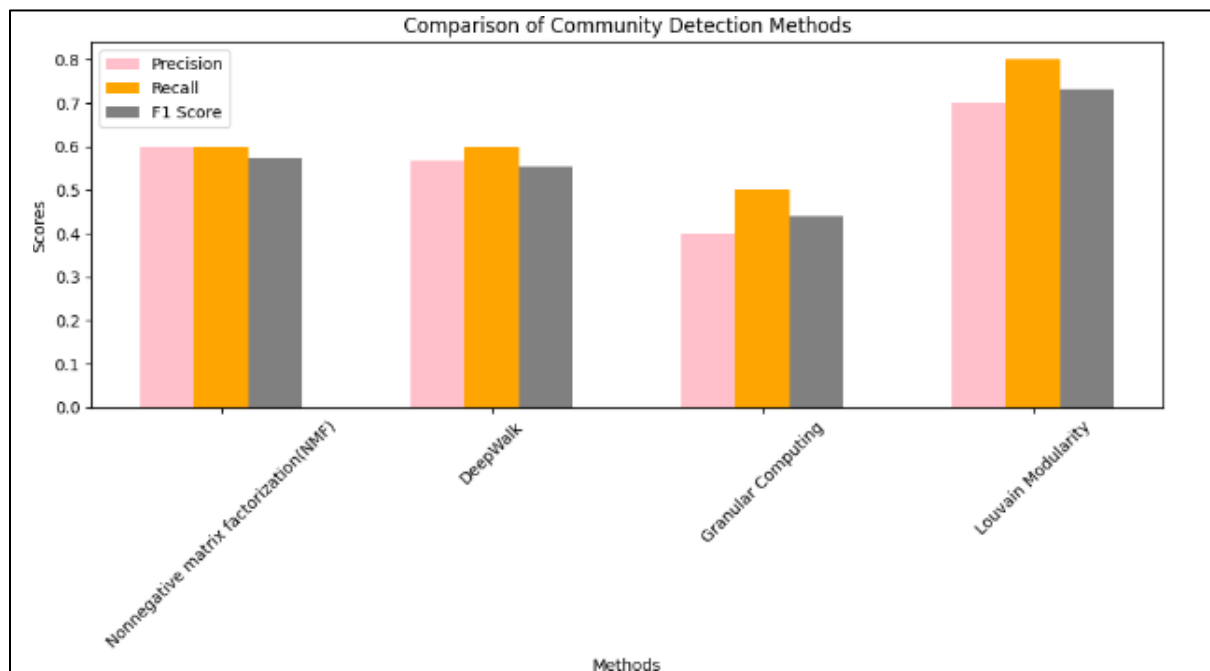


Figure 14 COMPARISON OF COMMUNITY DETECTION METHODS

Figure 15 shows the visualization and tabulation of evaluation metrics for various anomaly detection methods. The performance metrics of isolation forest algorithm is better than other methods infer that the anomalous nodes are detected more accurately using the isolation forest algorithm.

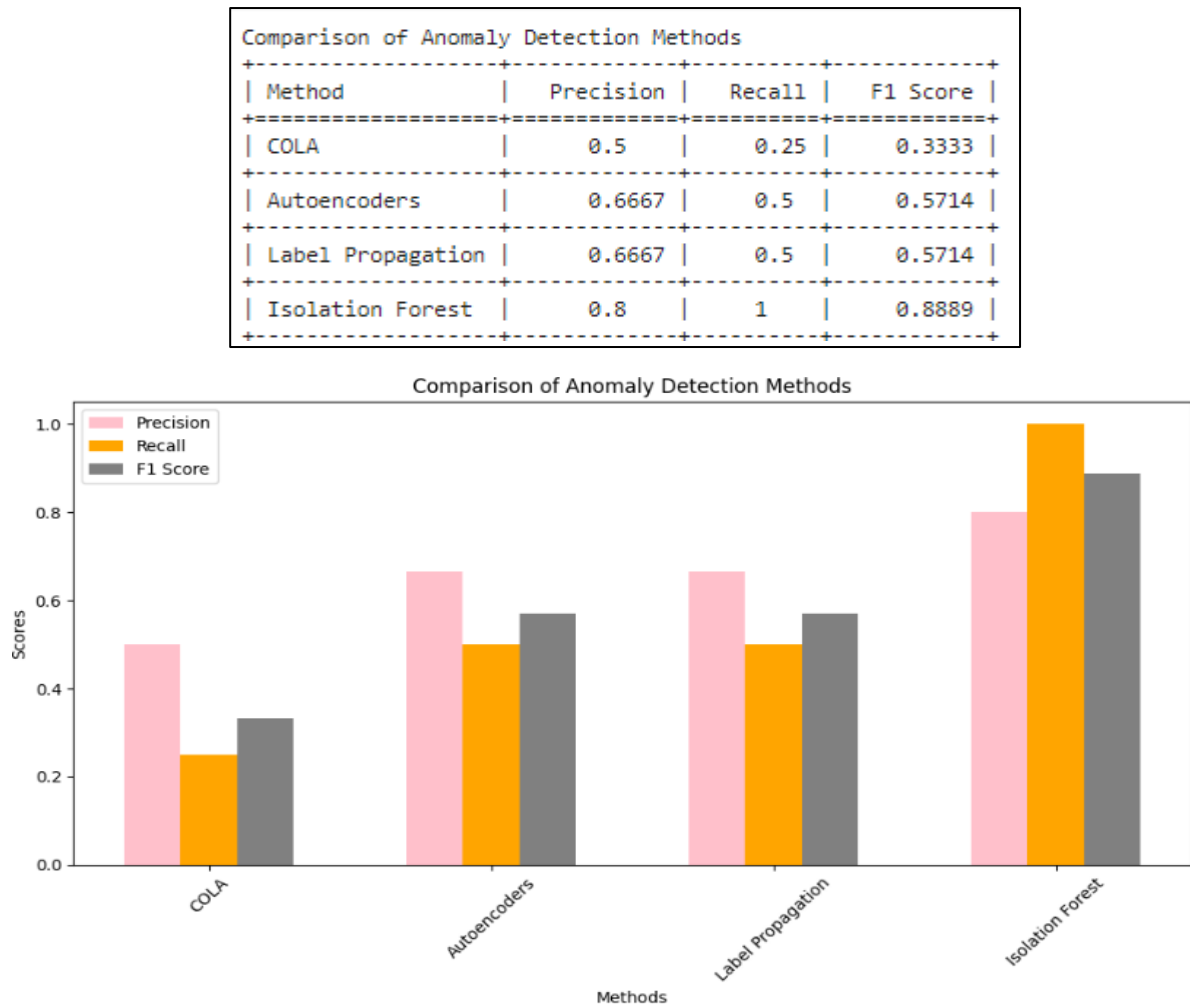


Figure 15 COMPARISON OF ANOMALY DETECTION METHODS

Figure 16 shows the visualization and tabulation of evaluation metrics for various influential user detection methods. The performance metrics of betweenness centrality algorithm is better than other methods infer that the influential nodes are detected more accurately using the betweenness centrality algorithm.

Comparison of Influential User Detection Methods			
Method	Precision	Recall	F1 Score
Change Point Detection	0.6667	0.5	0.5714
Nonnegative matrix factorization(NMF)	0.6667	0.5	0.5714
Betweenness Centrality	0.8	1	0.8889

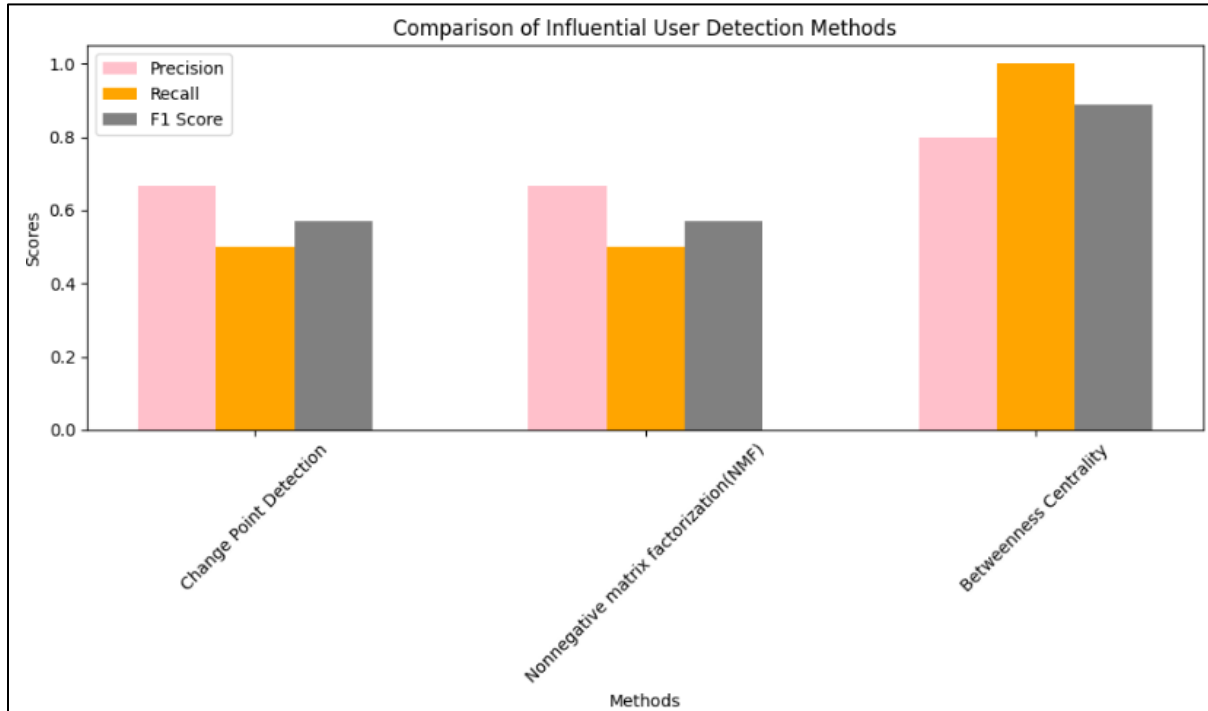


Figure 16 COMPARISON OF INFLUENTIAL USER DETECTION METHODS

5.3 SUMMARY

The implementation of the CLAID framework yielded promising results in analyzing social networks. It effectively identified closely related clusters through community detection using Louvain Modularity, detected suspicious behavior with Isolation Forest for anomaly detection, and pinpointed influential users using Betweenness Centrality. These insights into network structures, anomalies, and influence dynamics empower users with actionable intelligence for effective network management and decision-making.

CHAPTER 6

CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

In conclusion, CLAID framework represents a significant advancement in the analysis of social networks, addressing several key challenges prevalent in existing methods. By seamlessly integrating community detection, anomaly detection, and influential user identification into a unified approach, CLAID offers a brief understanding of social network dynamics, potentially leading to improved performance across all tasks. Moreover, the scalability of CLAID is enhanced through its integrated functionalities, eliminating the need for separate analyses and significantly reducing computational costs, particularly beneficial when dealing with large-scale networks. The utilization of the Louvain algorithm for community detection further contributes to computational efficiency by focusing on smaller communities rather than the entire network. Additionally, CLAID's capability to isolate anomaly nodes using the isolation forest algorithm enables the identification of suspicious activity within communities, enhancing the network's integrity.

Furthermore, CLAID's unified approach offers a steady workflow for social network analysis, simplifying the process for researchers and practitioners. By integrating community detection, anomaly detection, and influential user identification, CLAID reduces the complexity of conducting multiple analyses separately, saving time and resources. This streamlined workflow not only enhances efficiency but also encourages interdisciplinary collaboration, as researchers from various domains can leverage CLAID's comprehensive toolkit to gain deeper insights into social network structures and behaviours.

Additionally, the framework's flexibility allows for customization to specific use cases, enabling researchers to tailor analyses to the unique characteristics of different social networks, whether they are focused on online communities, organizational networks, or other social structures.

6.2 FUTURE WORKS

There are several exciting places for future research and development in the field of social network analysis, building upon the foundation laid by the CLAID framework. One potential direction is the exploration of dynamic networks, which evolve over time. While CLAID provides a solid framework for static network analysis, extending its capabilities to dynamic networks would enable researchers to capture temporal changes in social structures and behaviours, offering deeper insights into network dynamics and evolution.

Additionally, advanced future works for enhancing the CLAID framework could include integrating Graph Neural Networks (GNNs) to better analyze graph-structured social network data, developing algorithms for dynamic community detection to adapt to real-time changes, incorporating multimodal analysis to leverage diverse data types for a deeper understanding of user behavior, exploring privacy-preserving techniques such as differential privacy or federated learning, and conducting extensive real-world deployment and validation studies in collaboration with industry partners to refine CLAID based on practical performance and user feedback.

REFERENCES

- [1] Berahmand.K, M. Mohammadi, F. Saberi-Movahed, Y. Li and Y. Xu,” Graph Regularized Nonnegative Matrix Factorization for Community Detection in Attributed Networks,” in IEEE Transactions on Network Science and Engineering, vol. 10, no. 1, pp. 372-385, 1 Jan.-Feb. 2023, doi: 10.1109/TNSE.2022.3210233.
- [2] Chang.C.I,” Target-to-Anomaly Conversion for Hyperspectral Anomaly Detection,” in IEEE Transactions on Geoscience and Remote Sensing, vol. 60, pp. 1-28, 2022, Art no. 5540428, doi: 10.1109/TGRS.2022.3211696.
- [3] Elghanuni.R.H, M. A. M. Ali and M. B. Swidan,” An Overview of Anomaly Detection for Online Social Network,” 2019 IEEE 10th Control and System Graduate Research Colloquium (ICSGRC), Shah Alam, Malaysia, 2019, pp. 172-177, doi: 10.1109/ICSGRC.2019.8837066.
- [4] Folino.F and C. Pizzuti, ”Multiobjective Evolutionary Community Detection for Dynamic Networks,” in Proceedings of the IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining,2021
- [5] Hafiene.N, W. Karoui, L. B. Romdhane,” Influential nodes detection in dynamic social networks: A survey,” Expert Systems with Applications, vol. 159, p. 113642, 2020, ISSN 0957-4174, doi: 10.1016/j.eswa.2020.113642.
- [6] He.C, X. Fei, Q. Cheng, H. Li, Z. Hu and Y. Tang,” A Survey of Community Detection in Complex Networks Using Nonnegative Matrix

- Factorization,” in IEEE Transactions on Computational Social Systems, vol. 9, no. 2, pp. 440-457, April 2022, doi: 10.1109/TCSS.2021.3114419.
- [7] Huang et al.L,” Hybrid-Order Anomaly Detection on Attributed Networks,” in IEEE Transactions on Knowledge and Data Engineering, vol. 35, no. 12, pp. 12249-12263, 1 Dec. 2023, doi: 10.1109/TKDE.2021.3117842.
- [8] Huang et al.X,” A survey of community detection methods in multilayer networks,” in Data Mining and Knowledge Discovery, vol. 35, pp. 1–45, Jan. 2021, doi: 10.1007/s10618-020-00716-6.
- [9] Li.M, S. Lu, L. Zhang, Y. Zhang and B. Zhang,” A Community Detection Method for Social Network Based on Community Embedding,” in IEEE Transactions on Computational Social Systems, vol. 8, no. 2, pp. 308-318, April 2021, doi: 10.1109/TCSS.2021.3050397.
- [10] Li, X. Chen.Z, J. Song and J. Gao,” Adaptive Label Propagation for Group Anomaly Detection in Large-Scale Networks,” in IEEE Transactions on Knowledge and Data Engineering, vol. 35, no. 12, pp.12053-12067, 1 Dec. 2023, doi: 10.1109/TKDE.2022.3176478.
- [11] Liu.Y, Z. Li, S. Pan, C. Gong, C. Zhou and G. Karypis,” Anomaly Detection on Attributed Networks via Contrastive Self Supervised Learning,” in IEEE Transactions on Neural Networks and Learning Systems, vol. 33, no. 6, pp. 2378-2392, June 2022, doi: 10.1109/TNNLS.2021.3068344.
- [12] Luo.W, D. Zhang, L. Ni and N. Lu,” Multiscale Local Community Detection in Social Networks,” in IEEE Transactions on Knowledge and Data Engineering, vol. 33, no. 3, pp. 1102-1112, 1 March 2021, doi: 10.1109/TKDE.2019.2938173.

- [13] Luo.X, J. Wu, A. Beheshti, J. Yang, X. Zhang, Y. Wang, and S. Xue, "Coma: Community-Aware Attributed Graph Anomaly Detection," in Proceedings of the IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM),
- [14] Raj. E. D , G. Manogaran, G. Srivastava and Y. Wu,"Information Granulation-Based Community Detection for Social Networks," in IEEE Transactions on Computational Social Systems, vol. 8, no. 1, pp. 122-133, Feb. 2021, doi: 10.1109/TCSS.2019.2963247.
- [15] Sebastian.Y, J. T. Chew, X. T. Tiong, V. Raman, A. Y. Y. Fong, and P. H. H. Then," Anomaly Detection from Diabetes Similarity Graphs using Community Detection and Bayesian Techniques," in Proceedings of the IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)
- [16] Smith.A and B. Johnson, "Temporal dynamics of user influence in information cascades," Social Network Analysis and Mining, vol. 12, no. 1, pp. 1-15, January 2022. DOI: 10.1007/s13278-021-00702-3.
- [17] Souravlas et al.S," Probabilistic Community Detection in Social Networks," IEEE Access, vol. 11, pp. 18662-18672, 2023. doi: 10.1109/ACCESS.2023.3257021.
- [18] Su et al.X," A Comprehensive Survey on Community Detection with Deep Learning," in IEEE Transactions on Neural Networks and Learning Systems, doi: 10.1109/TNNLS.2021.3137396.
- [19] Tran.C, W.-Y. Shin, and A. Spitz,"Community Detection in Partially Observable Social Networks," in Proceedings of the IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining

- [20] Tu et al.C," A Unified Framework for Community Detection and Network Representation Learning," in IEEE Transactions on Knowledge and Data Engineering, vol. 31, no. 6, pp. 1051-1065, 1 June 2019, doi: 10.1109/TKDE.2018.2852958.
- [21] Wang.M, X. Zhao, and L. Li, "Modeling temporal dynamics of information cascades in social networks using recurrent neural networks," Neural Networks, vol. 123, pp. 183-195, May 2020. DOI: 10.1016/j.neunet.2019.11.017.
- [22] Wang.Y, C. Yang, D. Jin, and J. Dang, "Integrating Group Homophily and Individual Personality of Topics Can Better Model Network Communities," in Proceedings of the IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)
- [23] Wu.C, L. Zhang, and H. Wang, "Temporal intervention strategies for controlling information cascades in social networks," IEEE Transactions on Big Data, vol. 8, no. 2, pp. 365-378, April 2023. DOI: 10.1109/TBDATA.2022.3137396.
- [24] Zavrak.S and M. 'Iskefiyeli," Anomaly-Based Intrusion Detection from Network Flow Features Using Variational Autoencoder," IEEE Access, vol. 8, pp. 108348-108351, 2020. doi: 10.1109/ACCESS.2020.3001350.
- [25] Zhang.C, Y. Zhang, and B. Wu," A Parallel Community Detection Algorithm based on Incremental Clustering in Dynamic Network," in Proceedings of the 2018 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM), Beijing, China, 2018.
- [26] Zhang.G, Y. Liu, and J. Chen, "Detecting information cascades in social networks: A comprehensive review," IEEE Transactions on Knowledge

and Data Engineering, vol. 30, no. 6, pp. 1101-1114, June 2018. DOI: 10.1109/TKDE.2017.2789285.

- [27] Yang X, X. Zhang, Y. Yang, X. Chen and W. Tang, "GANDPS: Ensuring Both Detection Precision and Stability in GAN-based Anomaly Detection for Network Traffic," 2023 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom), Wuhan, China, 2023, pp. 305-310,doi:10.1109/ISPA-BDCloud-SocialCom-SustainCom59178.2023.00072.
- [28] Zhangbing Z., "Anomaly Detection in Social-Aware IoT Networks," in IEEE Transactions on Network and Service Management, vol. 20, no. 3, pp. 3162-3176, Sept. 2023, doi: 10.1109/TNSM.2023.3242320.
- [29] Zhu, P. Li, L. Yu, K. Chen and Y. Chen," Change Point Detection in Dynamic Networks Based on Community Identification," in IEEE Transactions on Network Science and Engineering, vol. 7, no. 3, pp. 2067-2077, 1 July-Sept. 2020, doi: 10.1109/TNSE.2020.2973328.
- [30] Zomaya A.Y, V. Sharma, R. KUMAR, W. -H. Cheng, M. Atiquzzaman, K. Srinivasan, "NHAD: Neuro-Fuzzy Based Horizontal Anomaly Detection in Online Social Networks," in IEEE Transactions on Knowledge and Data Engineering, vol. 30, no. 11, pp. 2171-2184, 1 Nov. 2018, doi: 10.1109/TKDE.2018.2818163